

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τεχνολογίες Υπηρεσιών Λογισμικού

Τεχνολογίες Υπηρεσιών Λογισμικού

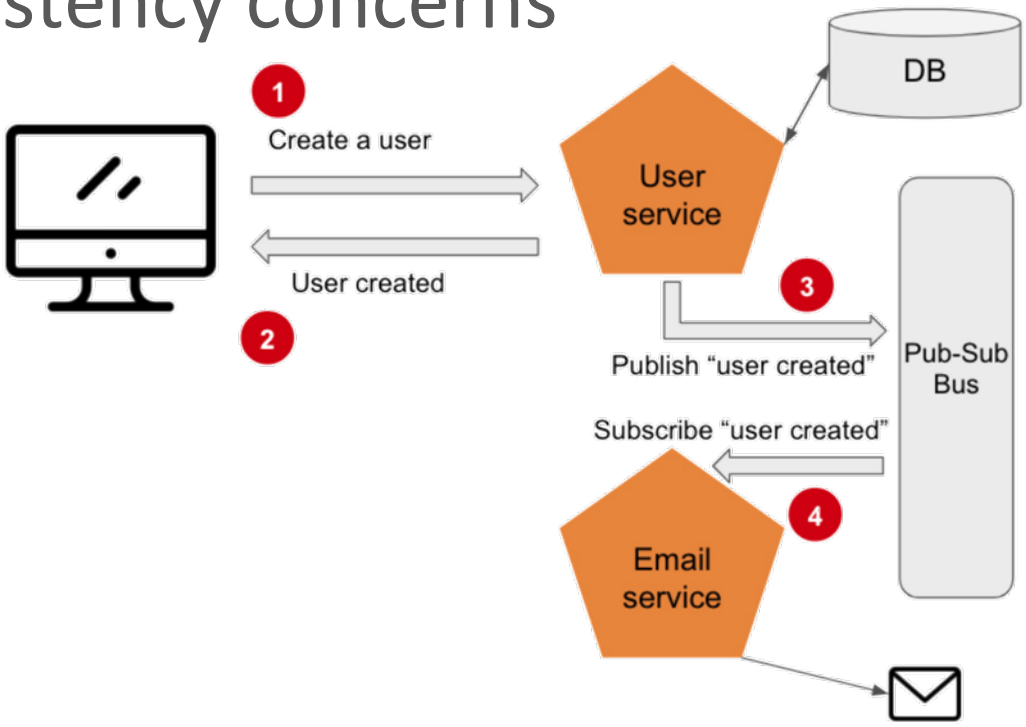
Β.Βεσκούκης, Καθ.ΕΜΠ (v.vescoukis@cs.ntua.gr)

Messaging

Inter-process communication

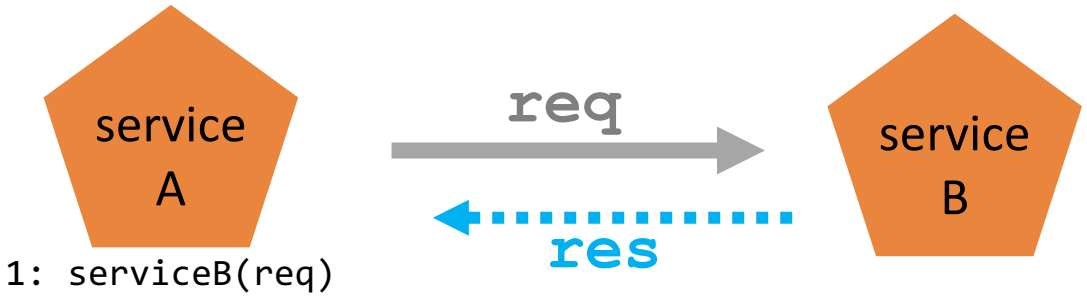
The publish-subscribe model in microservices choreography

- No direct or mediated communication
- Data consistency concerns

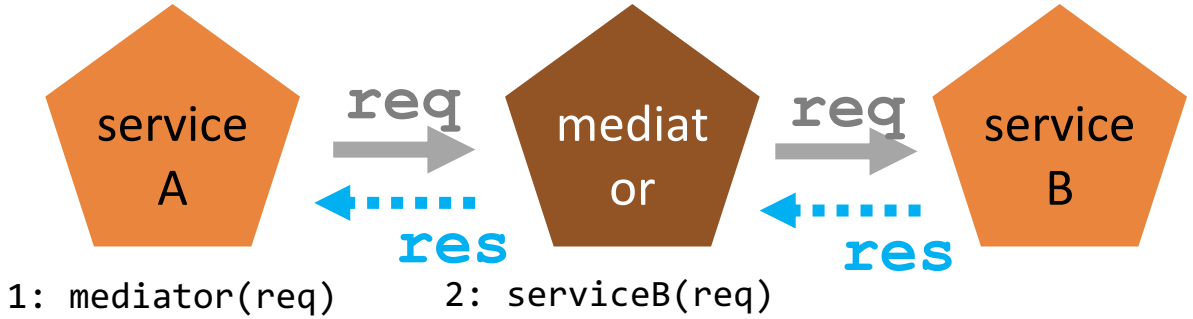


Inter-process communication

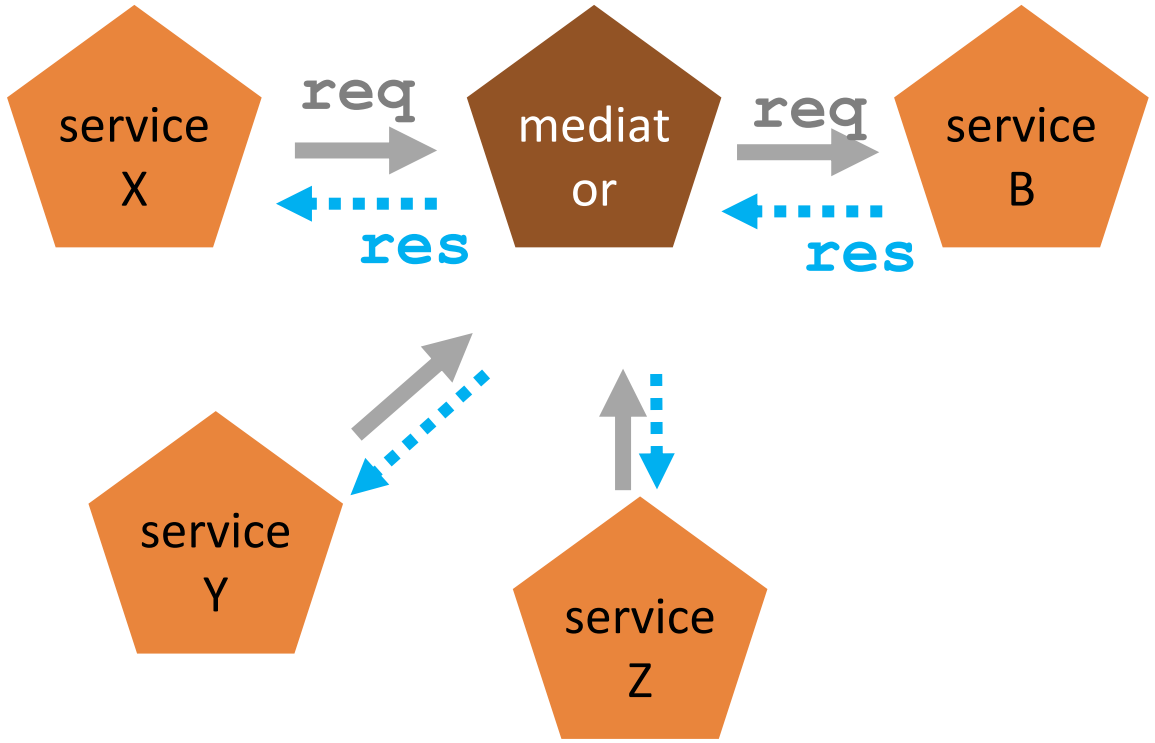
Direct communication



Mediated communication



Inter-process communication



Inter-process communication - http

http is synchronous



- **strong coupling**: clients need to wait for server response
- **propagation of delays**: one slow service delays all
- **poor scalability**: handling of many clients may need system restructuring
- **no retries** if request fails/delays
- does not handle **traffic bursts** (sudden peaks)

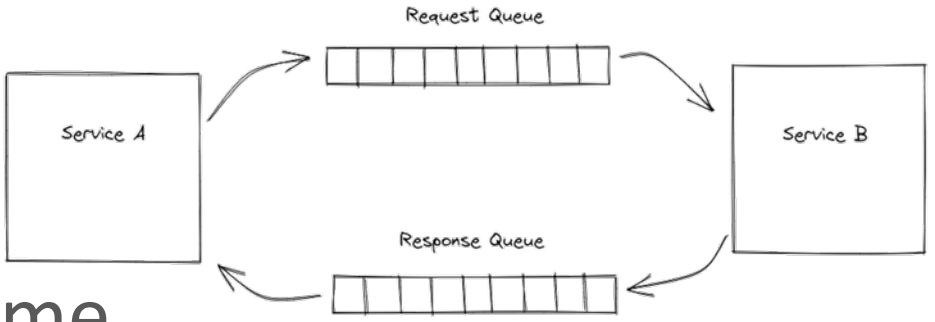
Asynchronous communication

No temporal-coupling between request, processing, response

- User performs a request and provides a callback to be notified when response is available.
- No data dependency:
  - run code that has no dependency to the response
  - schedule code with a hard dependency to the response, to be executed when possible
- Sender availability depends only on request time

Asynchronous communication

Request time depends on



- The network
- Service dependencies' lifetime
  - An async interface on ServiceA pushes the request to a separate queue service
  - ServiceB pulls from the queue, processes requests and then pushes responses to another queue
  - ServiceA service reads from the response queue and feeds the callback.

Asynchronous communication

What does this mean?

- instead of ServiceB we are now coupled to the queue's lifetime: the problem is not eliminated...
- Need to provide high-availability for the queue service (single point-of-failure)

- The cost of high-availability can be shared if the queue can be used by many services

Synchronous vs. asynchronous

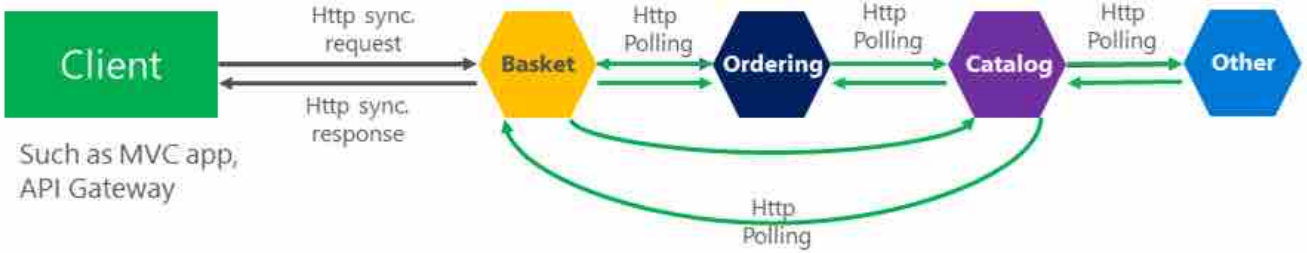
synchronous



async w/messaging

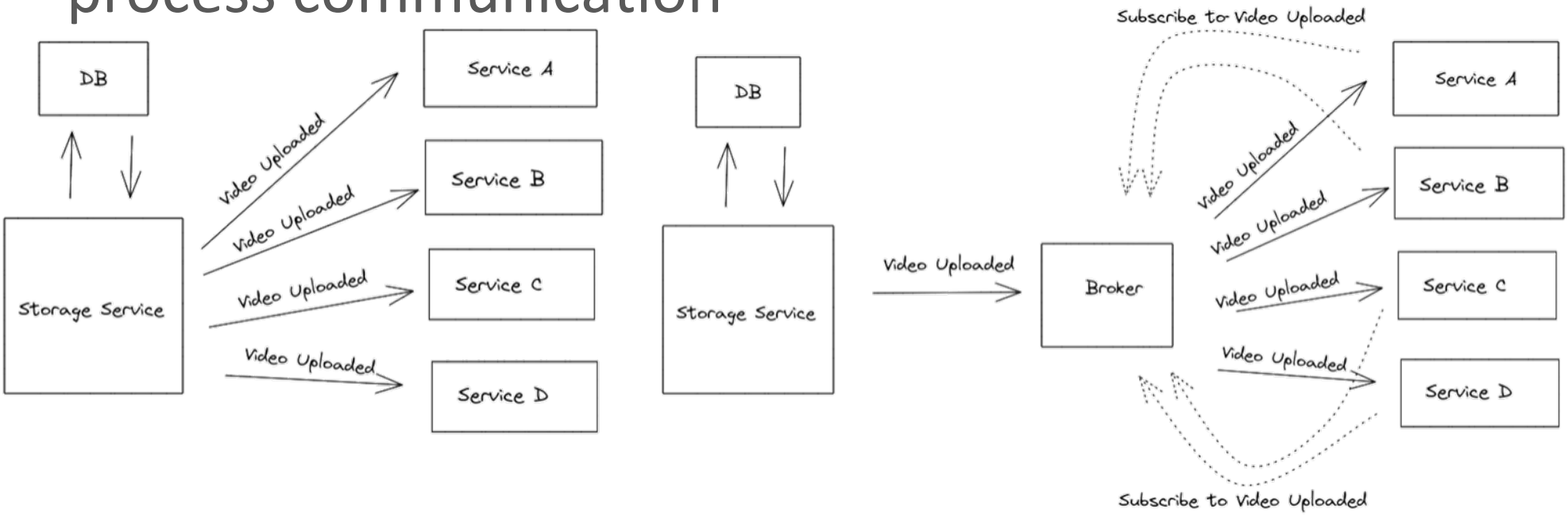


async over http

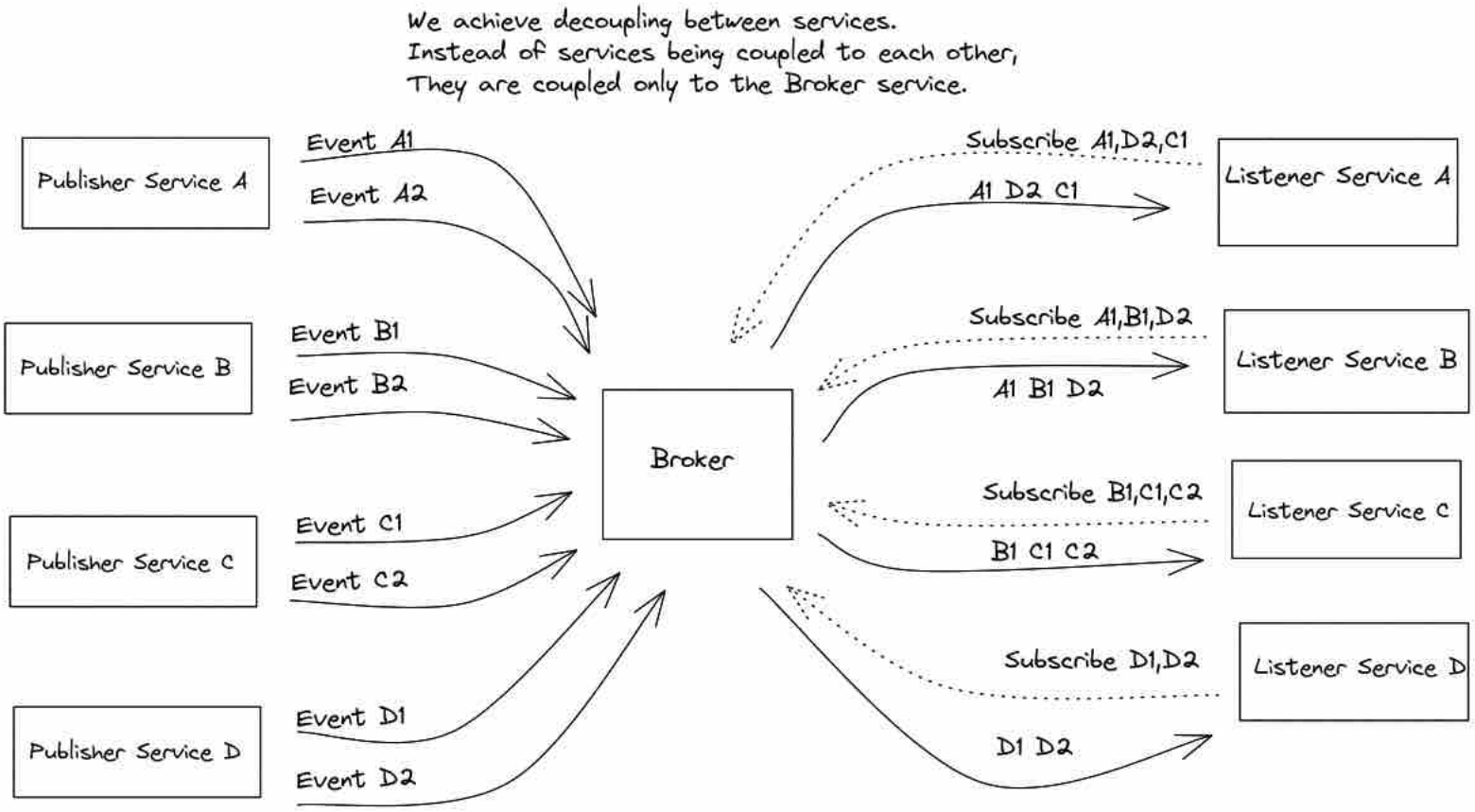


Events

- Messages can be more like notifications, acting as hooks that other service instances can use
- Such messages are called “Events” and they form the most flexible and decoupled way of inter-process communication

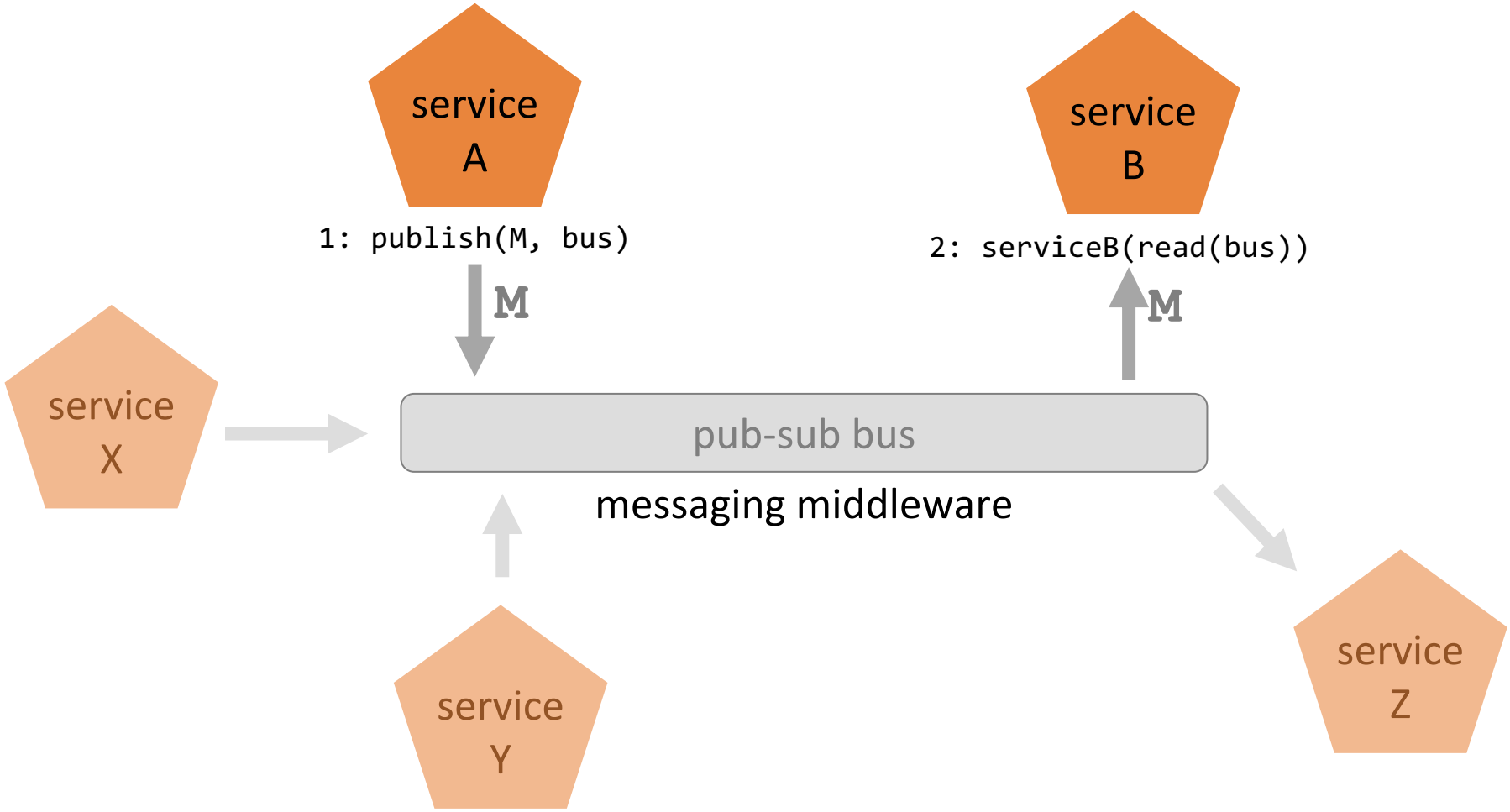


Events


We achieve decoupling between services.
Instead of services being coupled to each other,
They are coupled only to the Broker service.
Publisher Service A
Publisher Service B
Publisher Service C
Publisher Service D
Event A1
Event A2
Event B1
Event B2
Event C1
Event C2
Event D1
Event D2
Broker
Subscribe A1,D2,C1
A1 D2 C1
Subscribe A1,B1,D2
A1 B1 D2
Subscribe B1,C1,C2
B1 C1 C2
Subscribe D1,D2
D1 D2
Listener Service A
Listener Service B
Listener Service C
Listener Service D

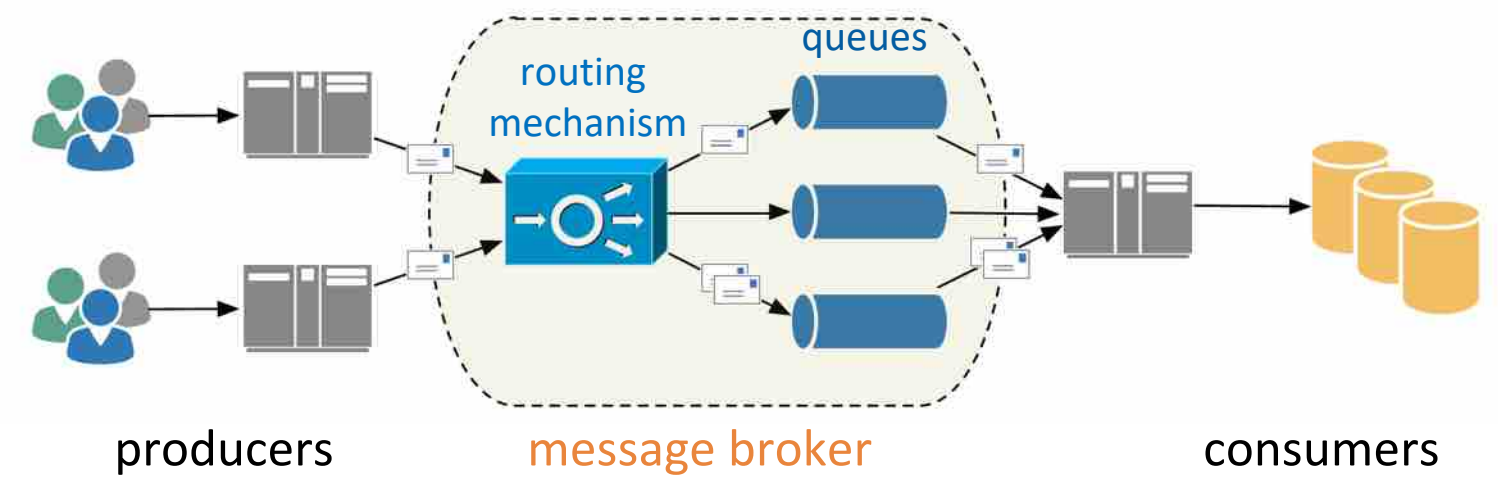
What is messaging?

Publish-subscribe bus



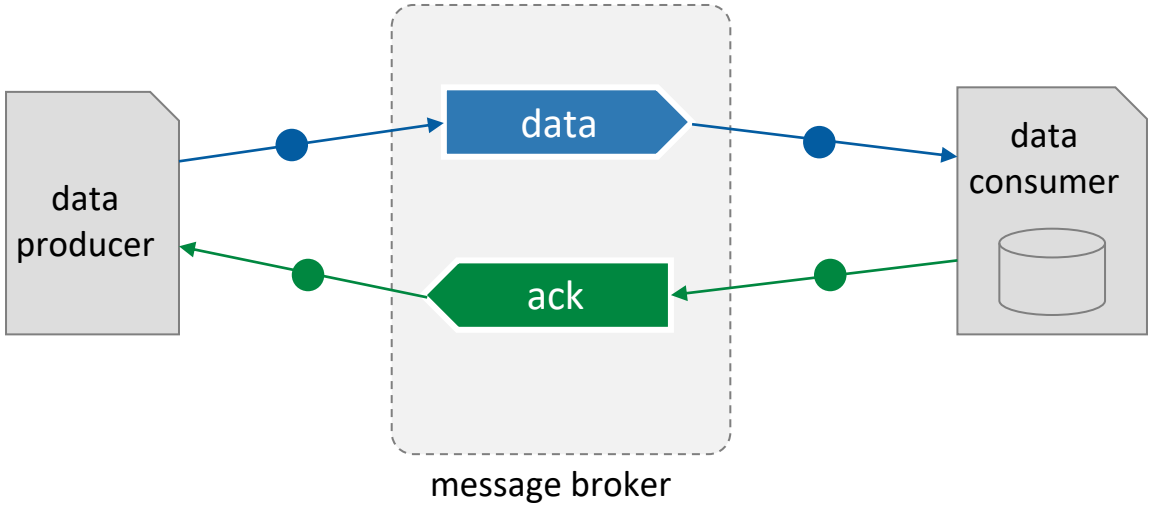
What is messaging?

Main objectives: scaling & extensibility



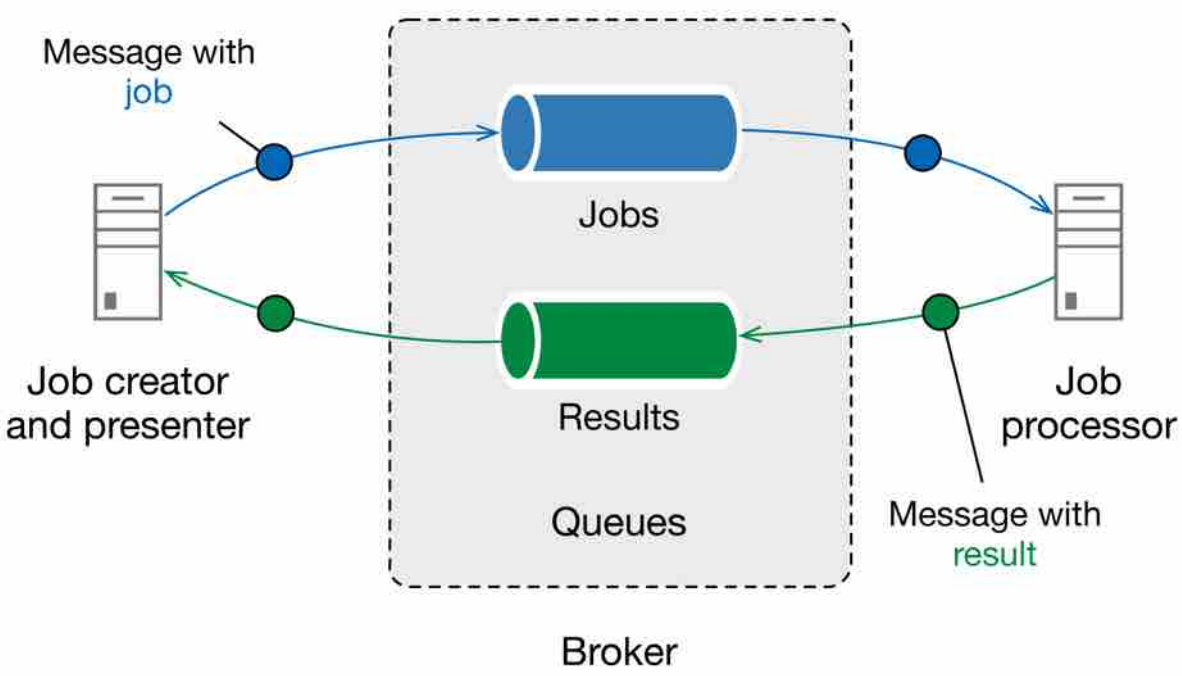
Messaging: use cases

Data communication

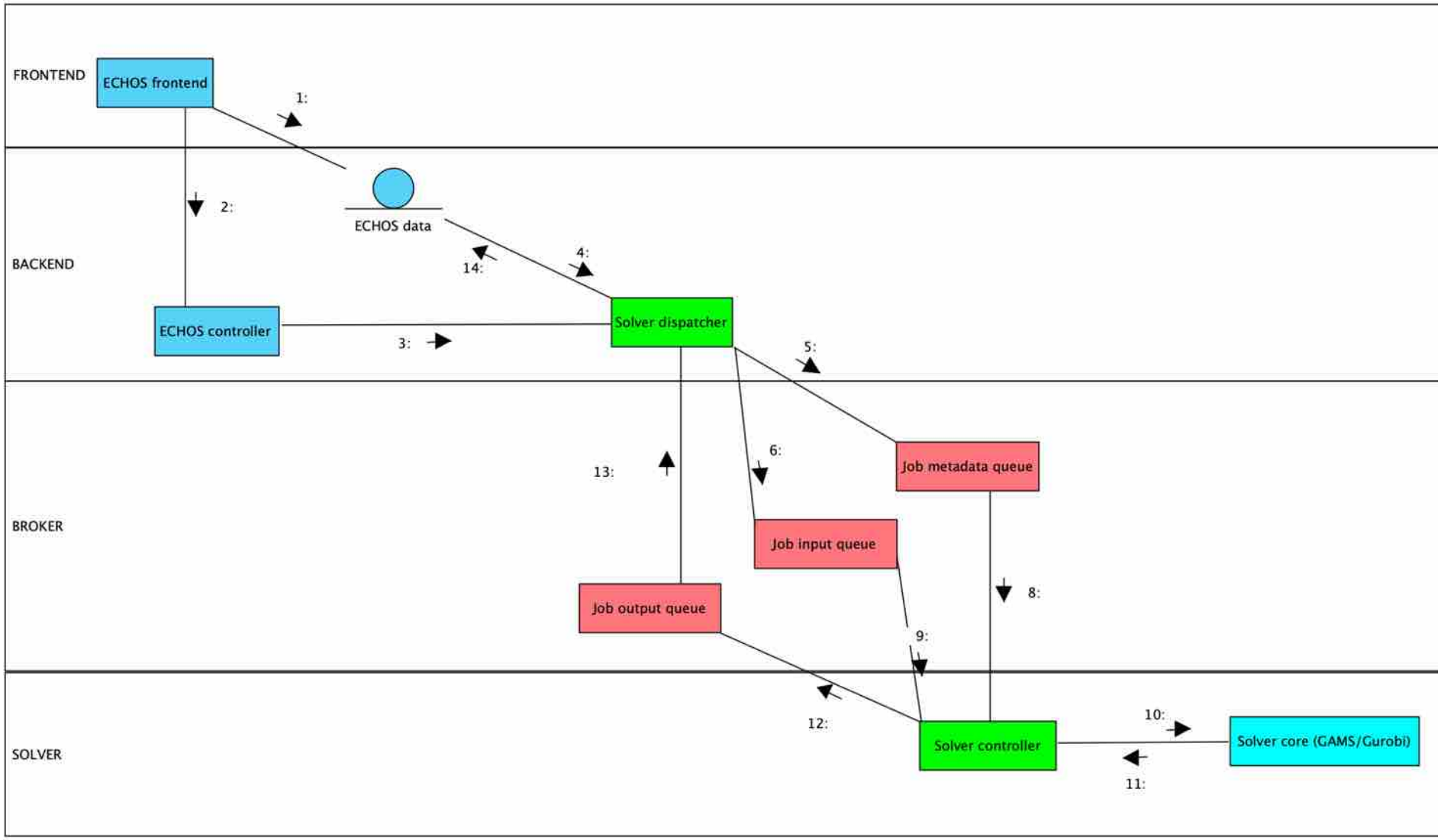


Messaging: use cases

Job request



Messaging - example (project 2023-2024)



Messaging middleware

Benefits

- Extensibility (add new message types)
- Scalability (add consumers/producers)
- Resilience, fault tolerance
- Process isolation

Challenges

- +1 service type to be managed
- Duplicated or lost messages, data consistency
- Delays, performance in general

Messaging middleware

Landscape

- IBM, Apache foundation, NEON (since early 2000's)
- JPMorgan Chase, Barclays, Cisco, Microsoft, VMware

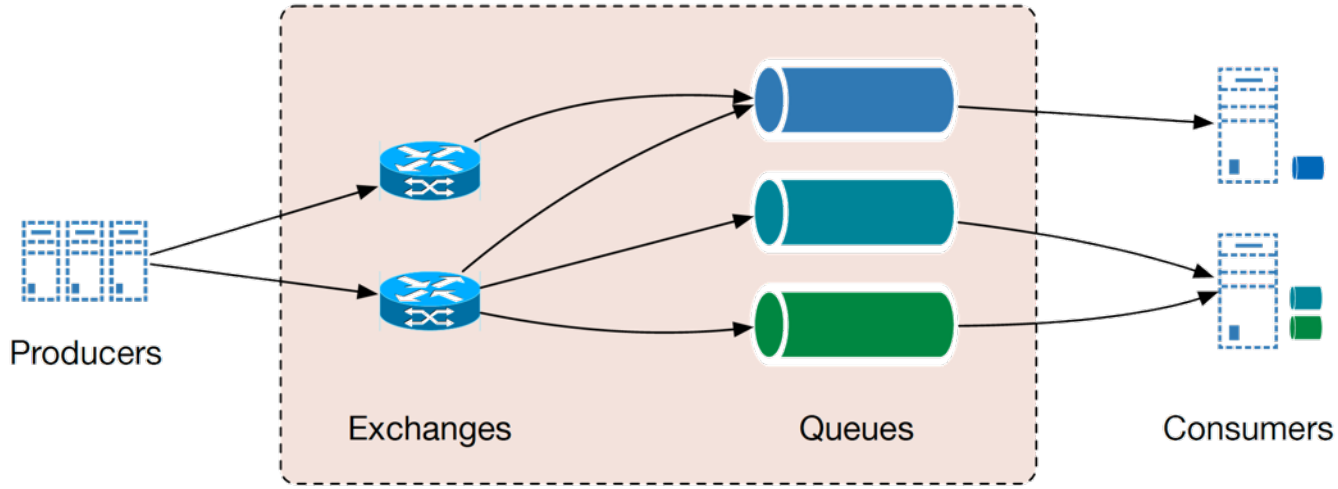
Standards

- AMPQ: Advanced Message Queuing Protocol (2011)
- STOMP: Simple Text-Oriented Messaging Protocol
- MQTT: Message Queue Telemetry Transport
- . . .

AMPQ messaging brokers

RabbitMQ

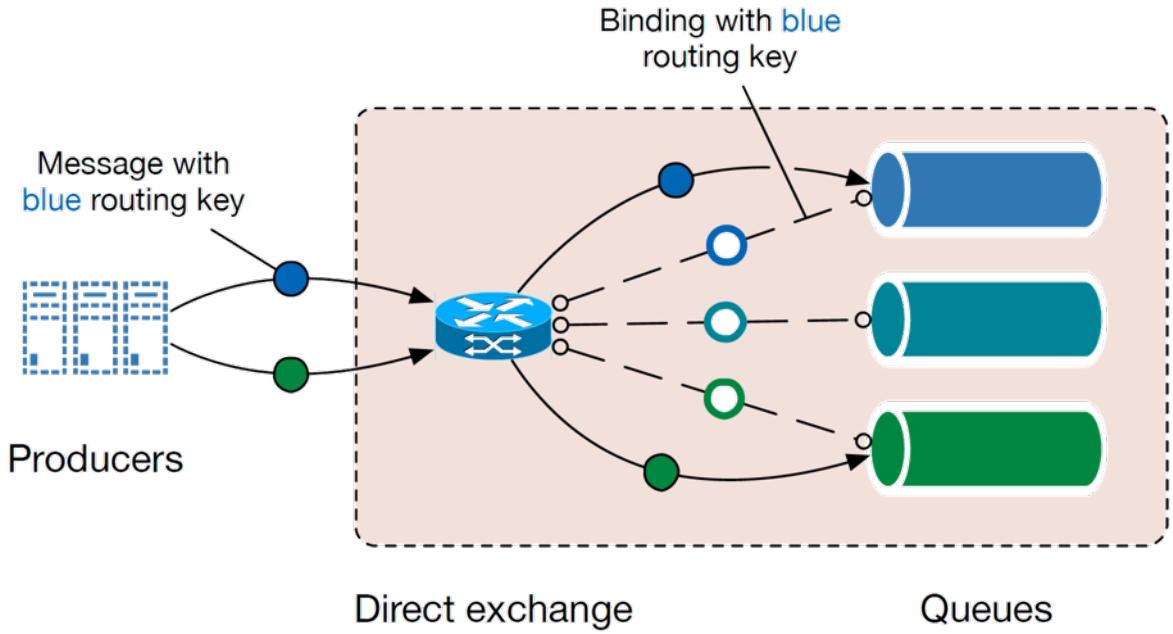
- Open-source
- Memory-based
- Stateless, not-ordered messages to consumers
- Routing rules implemented in "Exchanges"



RabbitMQ

Operation (direct exchange)

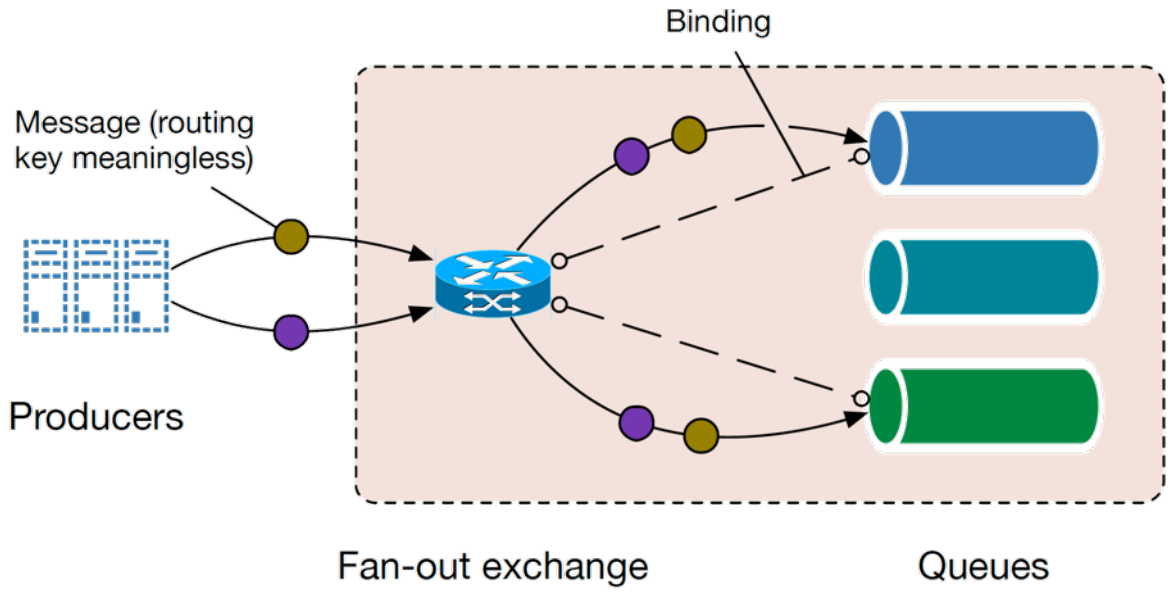
- Messages sent to exchanges
- Routing to queues determined by routing keys
- Rules sent each message to the designated queue



RabbitMQ

Operation (fan-out exchange)

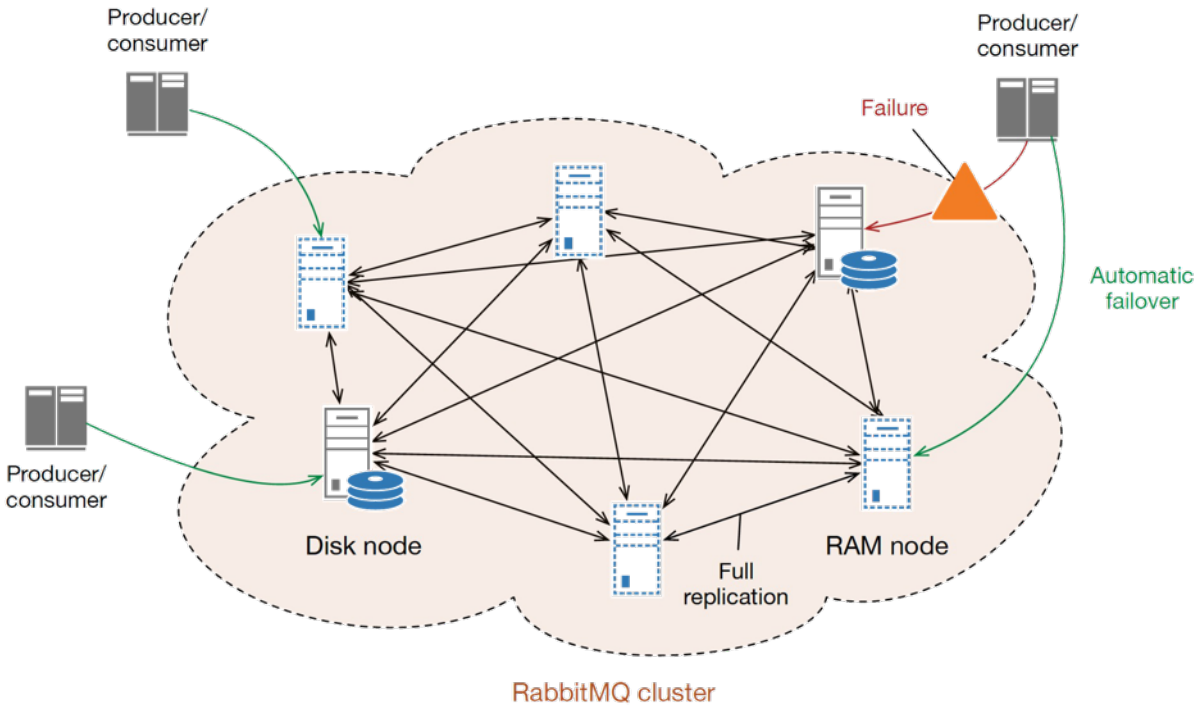
- Messages sent to exchanges
- From exchanges to all the queues served
- Useful in publisher-subscriber implementations



RabbitMQ

Key takeaways

- In-memory storage, optional limited disk storage
- Ack / rejection mechanism
- Dynamic queue and exchange creation
- Permission management & SSL
- Load balancing and replication features



Apache Kafka

Main features

- Open source, relies on Apache Zookeper, *"... a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services."*
- Implemented at LinkedIn (2011)
- Performance *and* persistence

Apache Kafka

2004-2010 – creation at Linkedin

- ...designed to "*provide a high-performance, scalable messaging system which could handle multiple consumers, many types of data [at high volumes and peaks], and provide for the availability & persistence of clean, structured data in real time*"

2011 – open source under the Apache Incubator

Apache Kafka

October 2012 – top project under Apache Software Foundation

2014 – several original Kafka engineers founded Confluent

- Introduction of Kafka Connect, Kafka Streams
- Offers Kafka as a service (Confluent cloud)

Kafka use cases

- Messaging & Queuing
- Handle fast data (IoT, social media, web clicks, infra metrics, …)
  - Receive and save – low latency, high volume
- Log aggregation
- Event Sourcing and Commit Log
- Stream processing
- Single enterprise event backbone
  - Connect business processes, applications, microservices

Kafka incarnations

Docker Images (Confluent)

Cloud:

- CloudKarafka
- IBM BlueMix Message Hub
- AWS supports Kafka
- Google runs Kafka

Kafka Connectors in many platforms

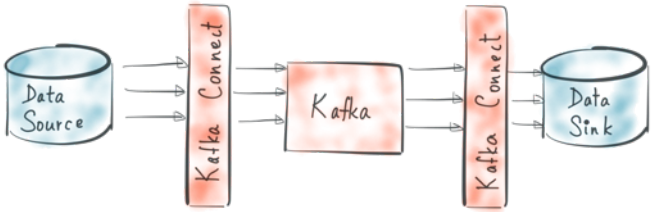
- Azure IoT Hub, Google Pub/Sub, etc

Kafka connectors

Kafka connects to anything...



- JDBC Source and Sink Connector
- Google BigQuery Sink Connector
- JMS Source Connector
- Elasticsearch Service Sink Connector
- Amazon S3 Sink Connector
- HDFS 2 Sink Connector
- MySQL Source (Debezium) Connector
- Replicator



Kafka terminology

Key terms

- Topic
- Message (key, value, timestamp)
- Broker (primary, replicas)
- Producer
- Consumer, consumer group

Kafka operation

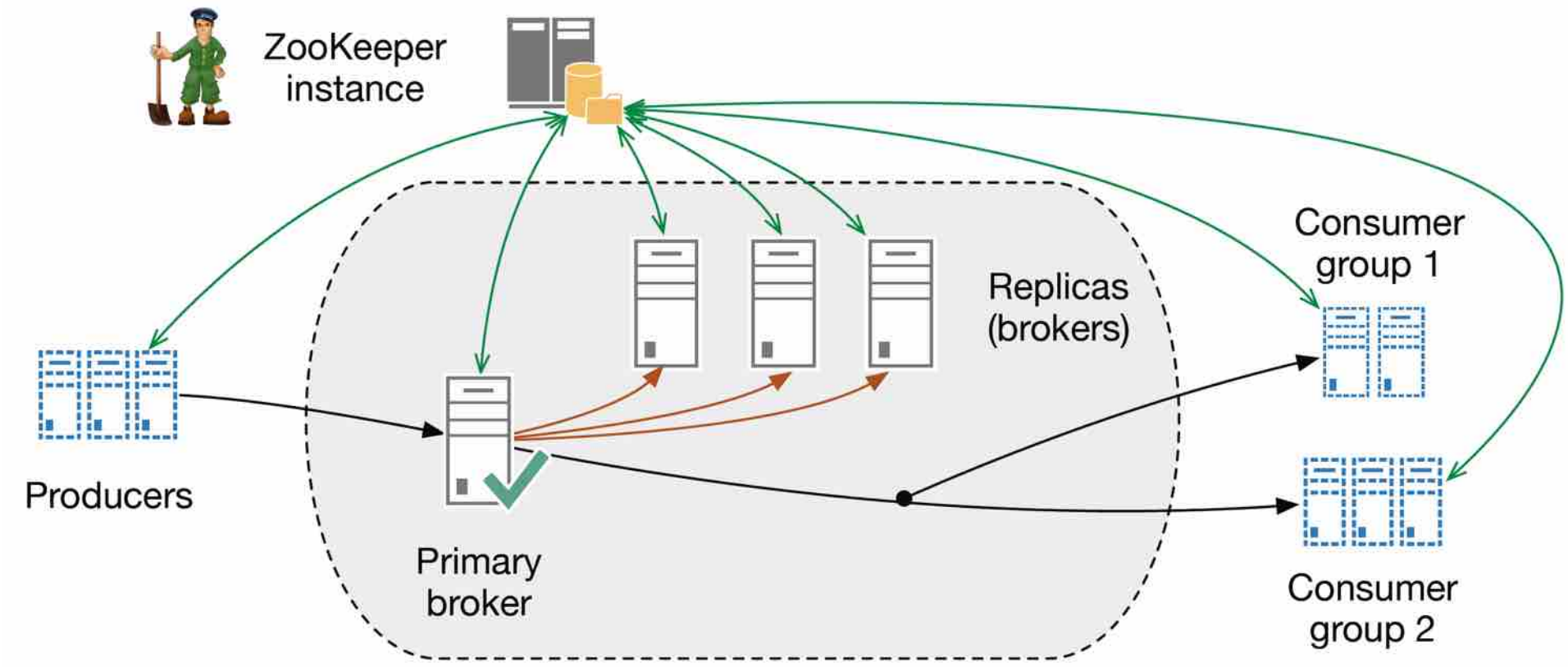
Operation principles

- Messages are available to consumers when they have been committed
  - "are available" = can be read
- Kafka does not push
  - the client has to ask for a message
- Read does not destroy
  - the message can be available after it has been read by consumers (this can change, if needed)

Kafka operation (cont'd)

- (some) History available
  - Offline consumers can catch up, not "indefinitely"
  - Consumers can re-consume from the (available) past
  - This behaviour can be tuned
- Delivery Guarantees
  - Ordering of messages is maintained
  - Ack at least once (per consumer) by default
  - "At-most-once" and "exactly-once" can be implemented

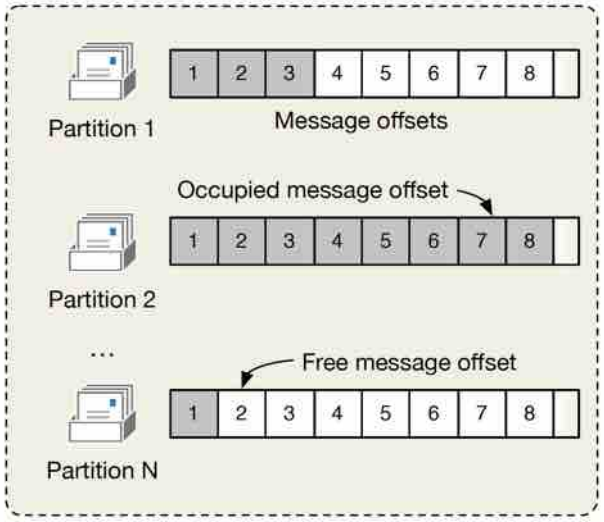
Apache Kafka



Apache Kafka: topics

Topic:

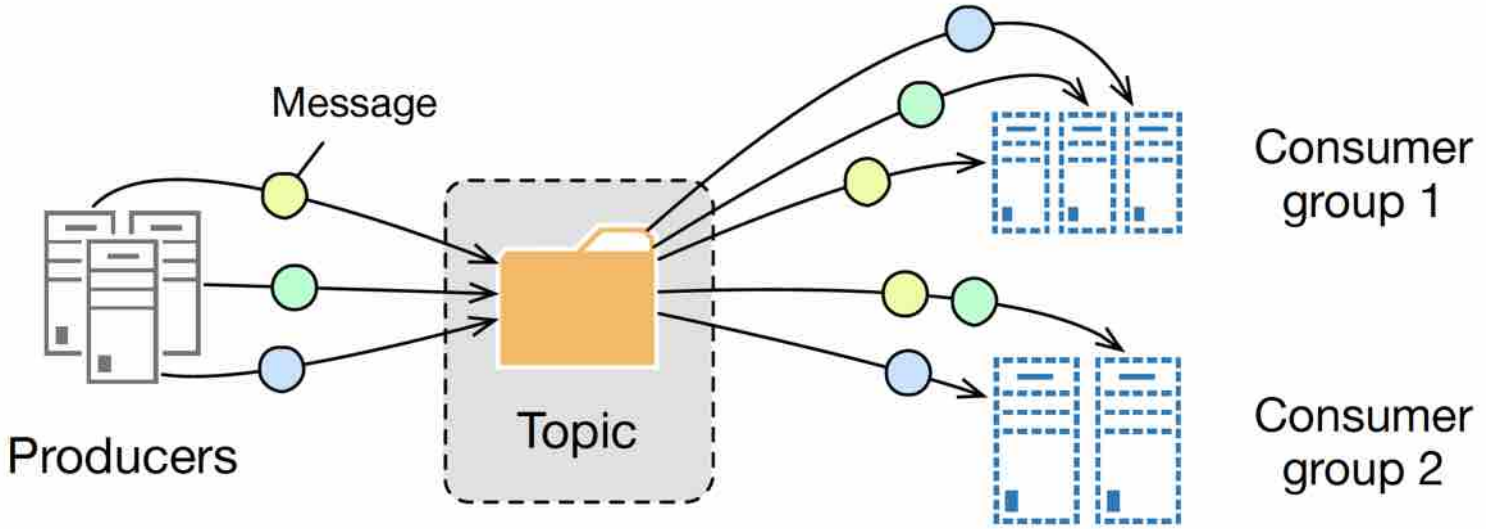
- a queue
- may be divided into partitions
- a broker may handle many topics
- messages are sent (and consumed) in order



Apache Kafka: consumer groups

Consumer group:

- one member only consumes a message in a topic
- different consumer groups implement ths publisher/subscriber functionality



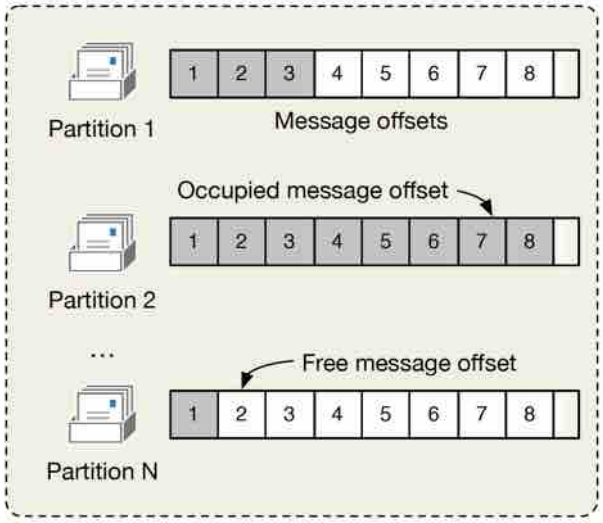
Apache Kafka

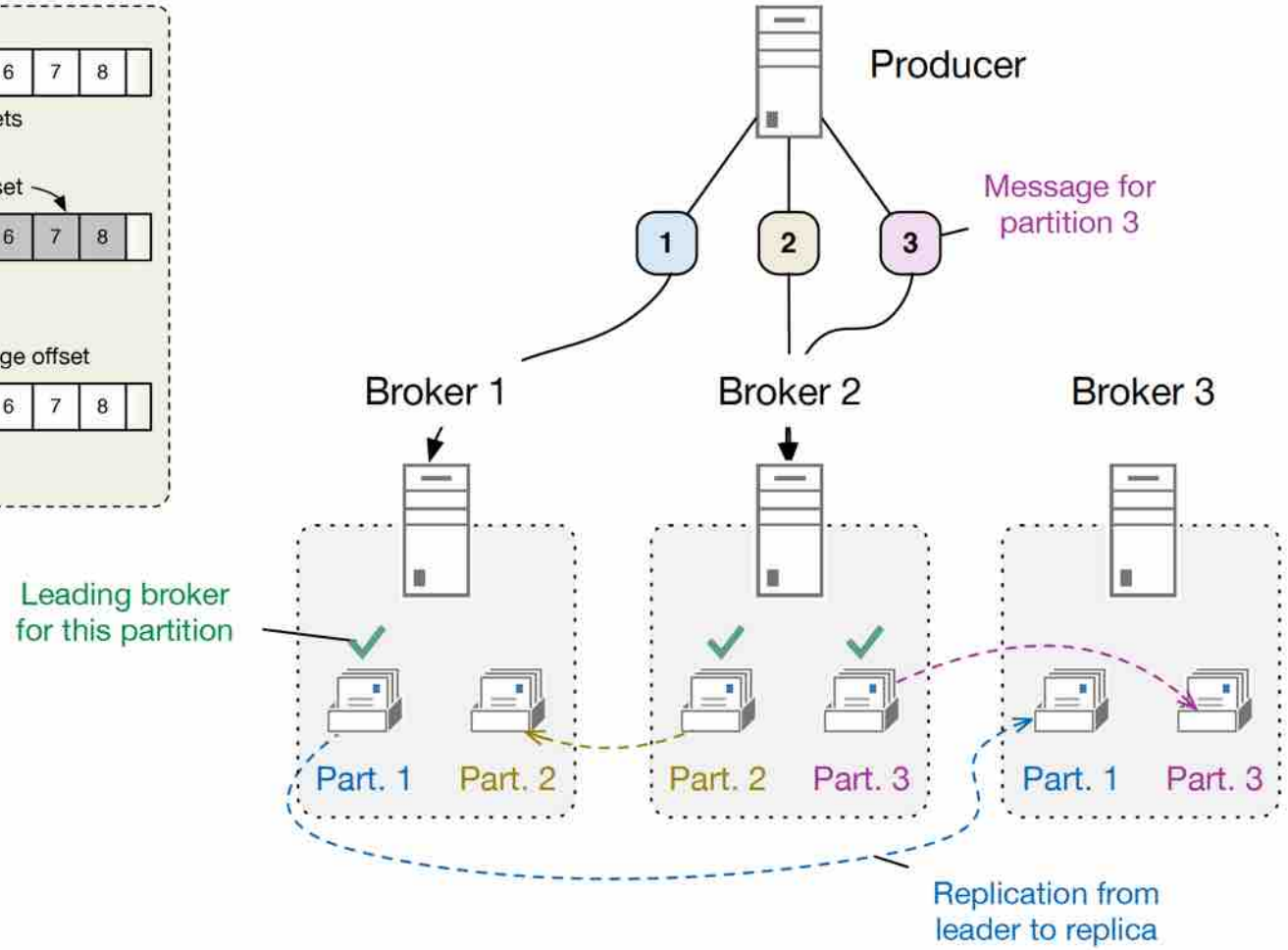
Key characteristics

- Queue persistence, no dynamic queues possible
- Per-partition load balancing and replication
- Asynchronous posting, tmp storage on producers
- Enqueuing ack
- Low & high-level consumer APIs
- Delete and compact mode for message archiving

Apache Kafka

Per-partition load balancing & replication

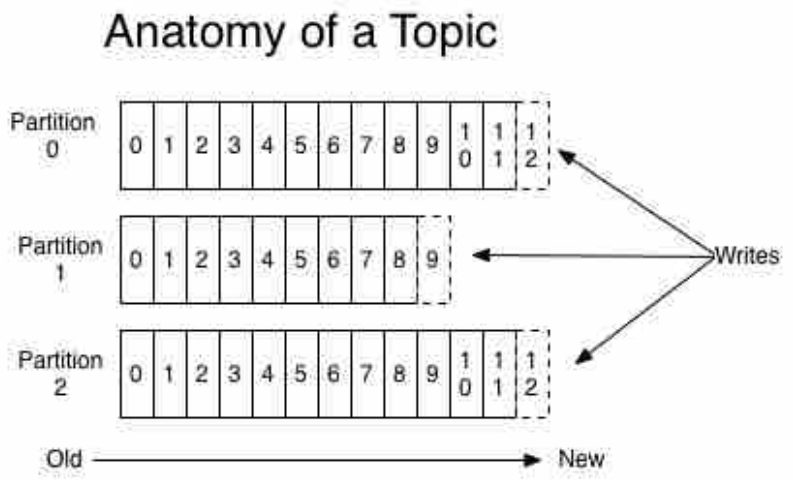




Partitions

Topics are configured with a number of partitions

- Storage, serialization, replication, availability, order guarantee are all at partition level
- Each partition is an ordered, immutable sequence of records that is continually appended to

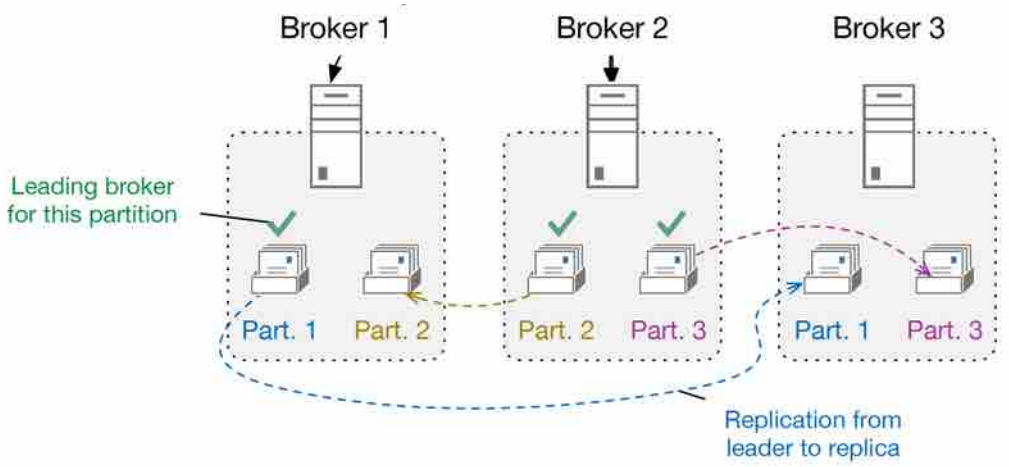


Partitions

Producer can specify the destination partition to write to

- Alternatively the partition is determined from the message key or simply by load balancing

Multiple partitions can be written to at the same time



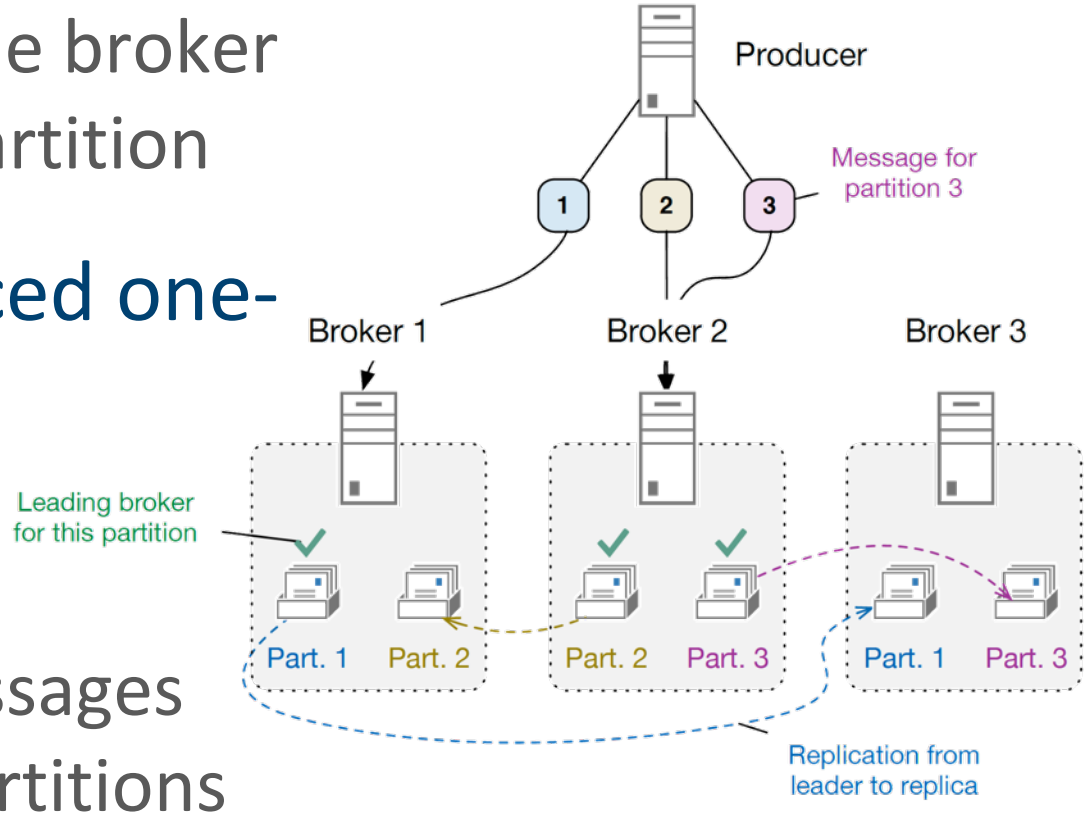
Producing messages

The producer sets the partition for each message

- Note: it should talk to the broker who is leader for that partition

Messages can be produced one-by-one or in batches

- Batches balance
- latency vs throughput
- A batch can contain messages for different topics & partitions



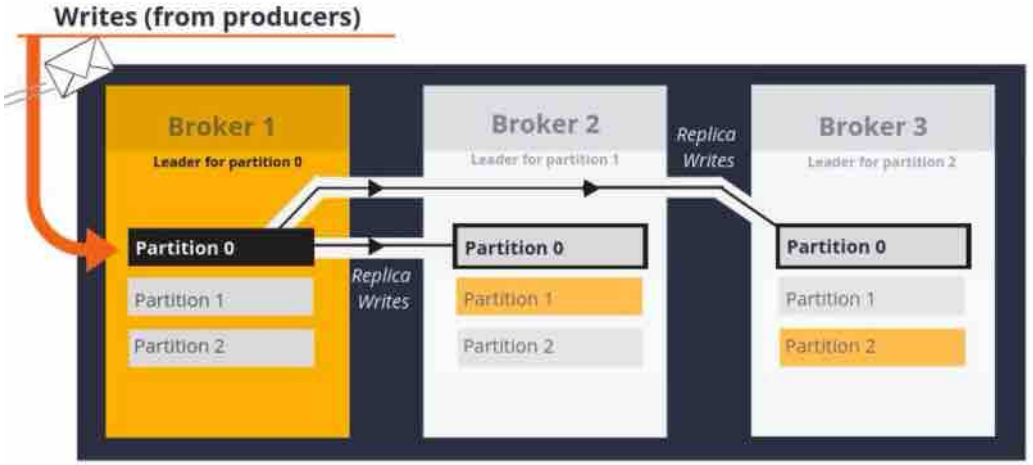
Producing messages

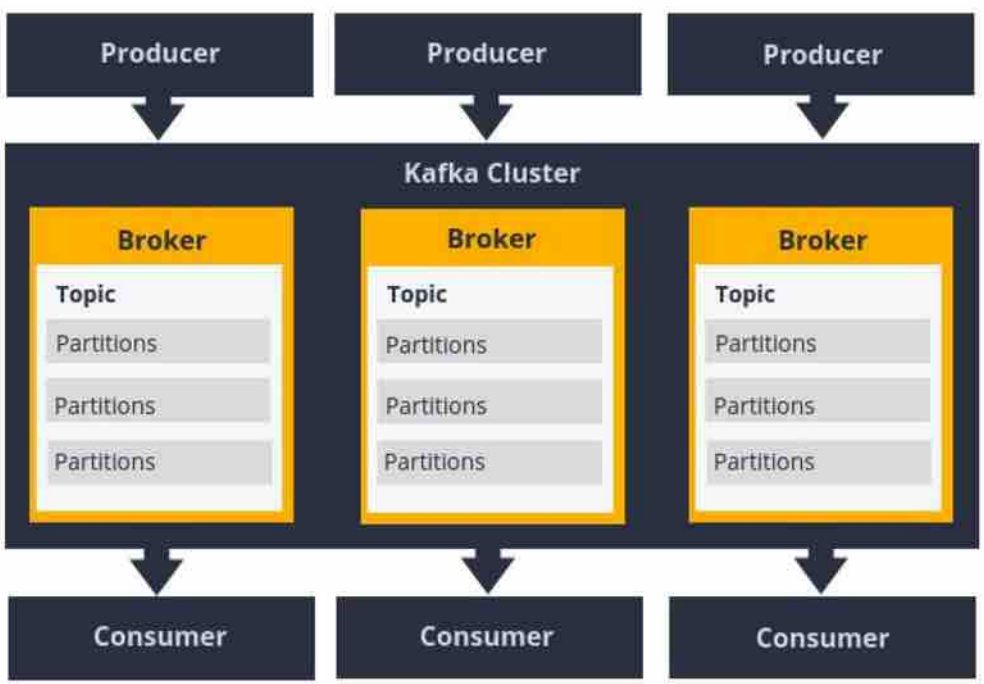
Messages can be compressed

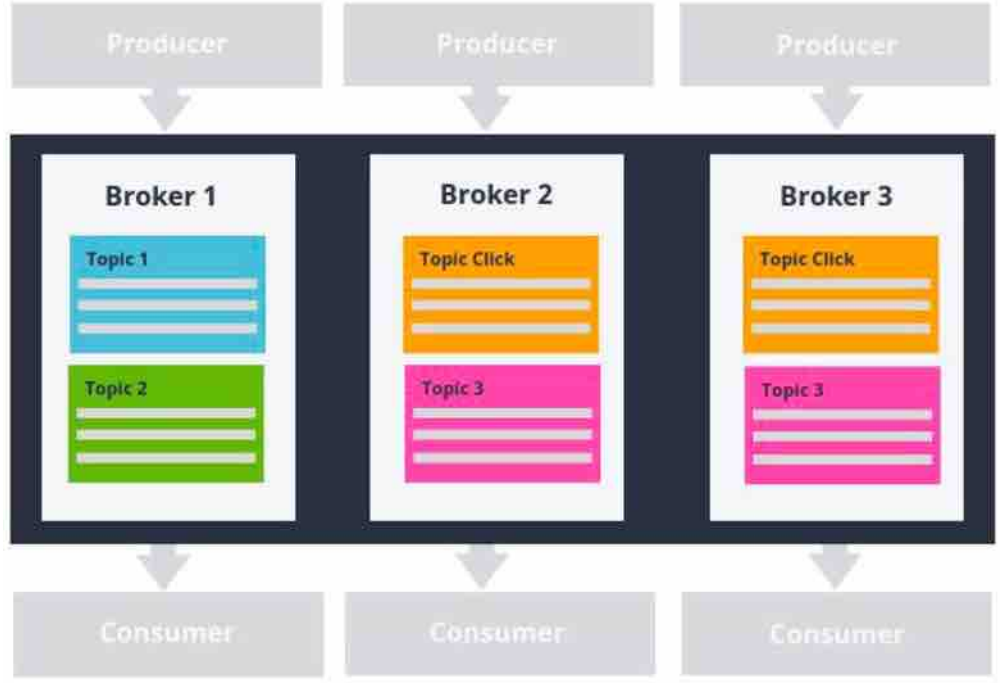
Producers can configure required ack level

- No (waiting for leader to complete)
- Wait for leader to commit [to file log]
- Wait for all replicas to complete

Note: messages are serialized to byte array format

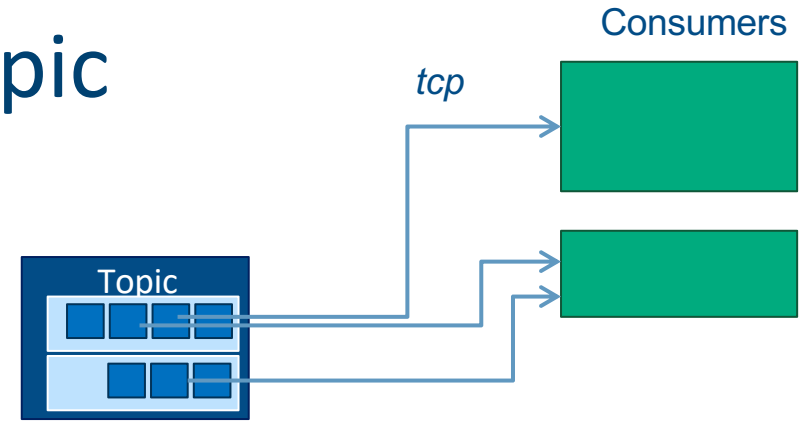



Producer
Producer
Producer
Kafka Cluster
Broker
Topic
Partitions
Partitions
Partitions
Broker
Topic
Partitions
Partitions
Partitions
Broker
Topic
Partitions
Partitions
Partitions
Consumer
Consumer
Consumer


Producer
Producer
Producer
Broker 1
Topic 1
Topic 2
Broker 2
Topic Click
Topic 3
Broker 3
Topic Click
Topic 3
Consumer
Consumer
Consumer

Consuming messages

A consumer pulls from a Topic



Consuming can be done in parallel to producing

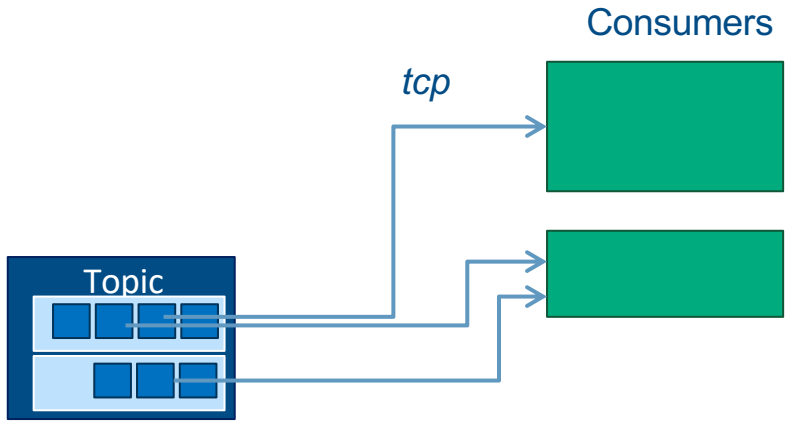
- Many consumers can consume at the same time

Each consumer has a Message Offset per partition, which...

- can be different across consumers
- can be adjusted at any time

Consuming messages

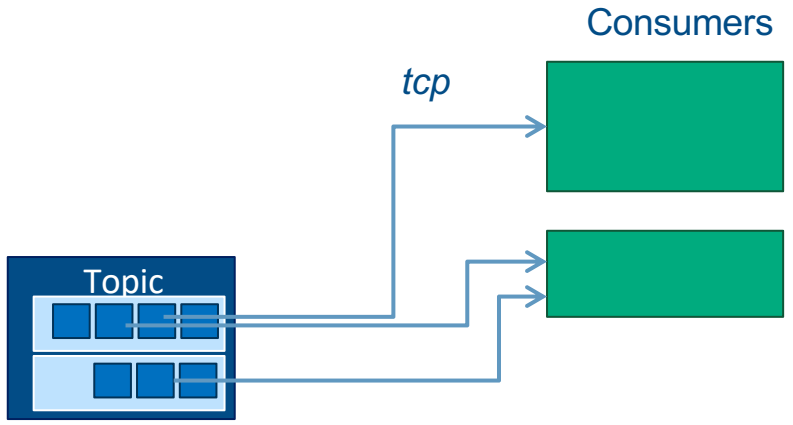
Delivery Guarantees



- At least once (per consumer)
  by default
- offset can be adjusted
  when all messages have been processed
- At-most-once and exactly-once
  can be implemented
  - Example: maintain offset in the same transaction that processes the messages)

Consuming messages

Message Retention



- Time Based
  (at least for … time)
- Size Based
  (log files can be no larger than … MB/GB/TB)
- Key based aka Log Compaction
  (retain at least the latest message for each primary key value)

What's next

Hands-on lecture

DIY

- https://www.rabbitmq.com/tutorials
- https://github.com/rabbitmq/rabbitmq-tutorials

- https://kafka.apache.org/quickstart
- https://developer.confluent.io/quickstart/kafka-local/

CREDITS

- Maarten Smeets & Lucas Jellema "Introducing Apache Kafka – scalable, reliable Event Bus & Message queue"
- Message-oriented Middleware for Scalable Data Analytics Architectures, N.Nannoni MSc thesis
- Apache Kafka beginners' guide, Elin Vinka, Lovisa Johansson
- Apache Kafka guide, Cloudera